

<u>RENAULT NISSAN DESIGN STANDARD</u> (RNDS) Part/Module generic specifications, containing RNDS-C-00068 v3.0	
<u>RENAULT STANDARD</u> (GDNormes) SPECIFICATION SHEET 36-00-029 --G	<u>NISSAN DESIGN SPECIFICATION</u> (NDS) SPECIFICATION 25953NDS32 [7]

Issued: 2018-03-12

<u>IMPORTANT PART SYMBOL</u>

Title: ECU programming. Boot-Loader mechanisms

Signature (Alliance Single Technical Leader) : Gabriel TOFFOLO

Signature (Renault Validator)

Gabriel TOFFOLO

Signature (Nissan Validator)

Shiro TSUJIOKA

Revisions

		DEA-SM	Modifications done for the package RFQ11 - Identity of person in charged updated - Creation of annex B for Reprogrammation of Multi CPU - Correction of a comment in the history chapter for the issue 2.0: "issue E" replaced by "issue --E/5" - Attribution of a responsible of validation for each requirements - Creation of REQ_DIAG_Reprogramming_4455 - REQ_DIAG_Reprogramming_4080 updated to take into account the case of one logical block with one segment. - Annex "A.4: Management of Boot Loader DTC type 0x06" updated - "Responsible for verification:" replaced by "Validation way" for all requirements - ECUReset (0x11) HW is optional. Modification of REQ_DIAG_Reprogramming_4300	3.0	--G	7
		DEA-SM	-§3 Modification of the Scratchpad's definition -§8 All requirement referenced in Annex B are modified -§7 "deleted requirements" created -REQ_DIAG_Reprogramming_4490 deleted -§6.5 updated with the new description of scratchpad -protection of the bootloader integrity : Creation of REQ_DIAG_Reprogramming_4035 and REQ_DIAG_Reprogramming_4021 -Addition of Initialization of the bootloader: modification of REQ_DIAG_Reprogramming_4220 and REQ_DIAG_Reprogramming_4230 -Correction of REQ_DIAG_Reprogramming_4400/4430: in the issue --E/5 of this document, these requirements were not completed transferred. They are corrected to be in accordance with the issue --D/4.	2.0	--F	6
G. TOFFOLO		DEA-SM	§3.2.1 Modification of comment for requirement REQ_DIAG_Reprogramming_4040 §3.3.2 Modification of Fig 3 to delete the requirement 0207 §3.3.2 Modification of the requirement REQ_DIAG_Reprogramming_4400 §3.3.2 Modification of the requirement REQ_DIAG_Reprogramming_4420	1.0	--E	5

N/A	N/A	N/A	2015.02.01	§Referenced Documents: Delete line "ISO Standards - ISO 14229 1:2013 (UDS)" §1 Remove sentences about interfaces between a programming tool and the boot-loader of an ECU §3.2.1 Add comment for requirement STD-PRG4-TS.0005 §3.2.2 Add example in STD-PRG4-TS.0010 comment §3.2.2 Move comment of STD-PRG4-TS.0011 in test of requirement §3.3.1 Figure 2 - Add pre-condition on STD-PRG4-TS.0107 §3.3.1 Modification of STD-PRG4-TS.0107 to add pre-condition before entering Boot-loader programming session §3.3.2 Modification of STD-PRG4-TS.0202 to indicate that digest has to be written outside of logical block and create requirement reference for remark §3.4 Modification of STD-PRG4-TS.0402 for deleting comment and change picture of figure 4. Annex A: Add Annex A.4 to complete STD-PRG4-TS.0005 §3.6 Add section about DELIVERY STATE OF ECU	N/A	--D	4
N/A	N/A	N/A	2014.04.01	§2.1 : Add comment for Digest in Glossary §3.3.1 : - Modification of programming session Initialization - Modification of STD-PRG4-TS.0106 / STD-PRG4-TS.0107 / STD-PRG4-TS.0108 to relocate response of request SDS programming session § 3.3.2 : - Add requirement STD-PRG4-TS.0220 - Add requirement STD-PRG4-TS.0221 Annex A : Add description for supported Failure type Annex B : Add description for conversion of Status for DTC	N/A	--C	3

N/A	N/A	N/A	2012.01.01	§ 3.3.2 : - Requirement STD-PRG4-TS.0201 : delete example of programming counter. - Requirement STD-PRG4-TS.0211 : addition of the consistency check (associated to logical block or global) - Figure 3: addition hardware reset as sub function for hard reset - Requirements STD-PRG4-TS.0207 and STD-PRG4-TS.0208 : Check coherency (FTB 0x06) for the logical block must be cleared. In the same way, Check coherency (FTB 0x06) for global DTC must be cleared.	N/A	--B	2
N/A	N/A	N/A	2011.01.01	- § 3.1 : New requirement STD-PRG4-TS.0302 about the service not impacted by the SECURITY_STATE parameter - § 3.2.1 : - requirement STD-PRG4-TS.0003 : "EACH LOGICAL BLOCK" instead of "SOME logical block" - requirement STD-PRG4-TS.0006 : Define a failure type for the General failure DTC - New requirement STD-PRG4-TS.0007 about refusing other DTC in boot-loader - § Figure 2: addition of a precision about the action after a reset - § Figure 3: addition of a precision about the standard session - Rename of 2nd requirement 212 with 213 - Modification of document reference in the top of the page	N/A	--A	1
N/A	N/A	N/A	2010.01.01	-included 0x06 failure	N/A	---	N
APPROVED BY	SIGN	SECT	APPROVED DATE	ALTERATION	CHG RNDS	Ver GDNormes	CHG NDS

RENAULT-NISSAN

Important Notices and Disclaimers

- "This document is identical to RENAULT / NISSAN document No. 36-00-029--G / 25953NDS32 [7]
It complies with the agreement reached between RENAULT and NISSAN in 2018-03-30.
Any revision or alteration of this document is subject to prior approval by the RENAULT or NISSAN secretariat.
- The original version of this document was written in English.
In the event of any discrepancies or differences in meaning created as a result of translation of this document into a foreign language, the meaning found in the English original shall take precedence.

Copyright notice

©RENAULT 2018

©NISSAN 2018

No duplication permitted without the consent of the issuing department.

No circulation permitted without the consent of RENAULT and NISSAN.

©RENAULT 2018

©NISSAN 2018

FOREWORD

This document complies with the agreement between RENAULT and NISSAN in June 2010 – March 2018 and is numbered **36 - 00 - 029** at RENAULT and **25953NDS32** at NISSAN.
Modifications must be agreed by the two companies.

RENAULT	Issued by	: Denis DESSERTENNE
	Validated by	: Gabriel TOFFOLO
NISSAN	Issued by	: Iichiro TOJO
	Validated by	: Shiro TSUJIOKA

Table of contents

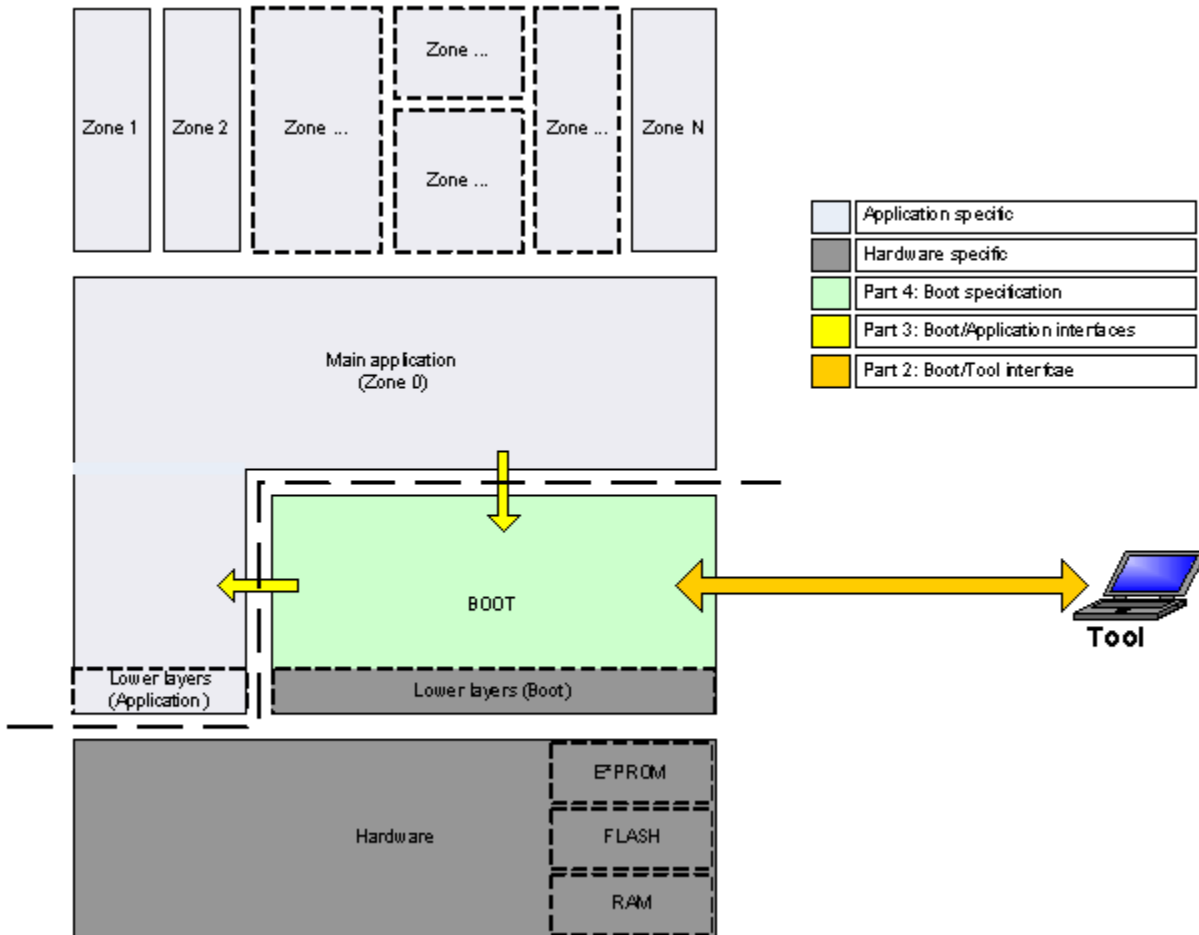
1	SCOPE	10
2	NORMATIVE REFERENCES	11
3	TERMS AND DEFINITIONS	12
4	SYMBOLS AND ABBREVIATIONS.....	13
5	REQUIREMENTS TERMINOLOGY	14
6	REQUIREMENTS	15
6.1	General	15
6.2	Boot-loader states	16
6.2.1	Boot-loader DTCs.....	16
6.2.2	Counters.....	18
6.2.3	Internal states	20
6.3	Boot-loader activity diagram	22
6.3.1	Entering Programming Session	23
6.3.2	Handling Programming procedures	26
6.4	Logical Block.....	31
6.5	Scratchpad and Memory Handler	32
6.6	Delivery State of ECU.....	34
7	DELETED REQUIREMENTS	37
8	Annex A: Supported Failure type.....	38
8.1	A.1: Supported Failure type for ECU with ONLY one logical block	38
8.2	A.2: Supported Failure type for ECU with multiple logical block and no global DTC	38
8.3	A.3: Supported Failure type for ECU with multiple logical block and global DTC	39
8.4	A.4: Management of Boot Loader DTC type 0x06	39
9	Annex B: conversion of the DTC status.....	40
9.1	B.1: Management of DTC with FTB 0x51 or 0x06.....	40
9.2	B.2: Management of DTC with FTB 0x42	40
10	Annex C: Memory Handler management	42
10.1	C1: 1 st case ECU does not have scratchpad	42
10.2	C2: 2 nd case ECU has scratchpad	43
11	Annex D (Informative): Reprogrammation of Multi CPU	48
11.1	D.1 Use cases to apply rules on multi CPU	48
11.2	D.2 Reprogrammation rules.....	48
11.3	D.3 Sequencing of transfer data.....	50
11.3.1	D.3.1 Sequential transfer	51
11.3.2	D.3.2 Example of the reprogrammation of a Multi CPU ECU: 2 ECU/ 4 logical blocks....	51
11.4	D.4 After reprogrammation	52
11.5	D.5 DID management.....	52

INTRODUCTION

This document is a part of Part 4 - Bootloader mechanism specification package. This package is divided into five parts, consistent to each others.

- Part 1: General description (Nissan ref : **25953NDS29** /// Renault Ref **36-00-027**)
- Part 2: Boot/tool interfaces description (Nissan ref : **25953NDS30** /// Renault Ref **36-00-028**)
- Part 3: Boot/Application interface description Nissan ref: **25953NDS31** /// Renault Ref **36-00-033**)
- Part 4: Boot specification (Nissan ref : **25953NDS32** /// Renault Ref **36-00-029**)
- Part 5: Conformance test (Nissan ref : **25953NDS33** /// Renault Ref **36-00-031**)

Figure



The Part 4 - Bootloader mechanism package contains specification on boot, how it interfere with application (application reprogramming and launch), with tool (how to reprogram an application, conformity test) and means to test the boot.

Boot-loader interfaces are:

Table (1/3)

	Name	Comment
Boot/Application		

Table (2/3)

BootTool	BOOT_PROG_SESSION	
	BOOT_SA_UNLOCK_REQ	
	BOOT_WRITE_DIGEST	
	BOOT_REQ_DOWNLOAD	
	BOOT_TRANSFER_DATA	
	BOOT_RUNDTCTEST	
	BOOT_GET_LOGICALBLOCK_INFO	

Table (3/3)

Boot/Lower layer		

1 SCOPE

This document is applicable for Renault in DEA – SMD3 and for Nissan in XX6.

2 NORMATIVE REFERENCES

	[RENAULT]	[NISSAN]	[ALLIANCE]
[REF 1] UDS Implementation.	36-00-011	25953NDS27	RNDS-C-00065
[REF 2] Rules for the management of on-board data identifiers in UDS implementation.	36-02-014	25953NDS28	RNDS-C-00073

Title	Ref.	Rev.
[1] Road vehicles — Unified diagnostic services (UDS) — Part 1: Specification and requirements	ISO 14229-1	2013
[2] Road vehicles — Diagnostics on Controller Area Networks (CAN) — Part 3: Implementation of unified diagnostic services (UDS on CAN)	ISO 15765-3	2004
[3] Road vehicles — Communication between vehicle and external equipment for emissions-related diagnostics — Part 6: Diagnostic trouble code definitions	ISO 15031-6	2005

3 TERMS AND DEFINITIONS

Segment

A segment is in memory contiguous part of data.

Logical Block

A logical block is a block of consistent data that can be unitarily be reprogrammed (e.g. calibration or software module). A logical block is built of one or more segments.

Unlocking fingerprint

This is a data that represent the entity for which securityAccess has been allowed.

Digest

Result of a cryptographic hash function. It represents the calculated signature of a data set. The supplier is free to choose cryptographic hash function to calculate Digest. SHA1 or MD5 will be used for security access so it's possible for reducing development cost/delay to use SHA-1 or MD5 to calculate Digest.

Scratchpad

Volatile memory part that can be programmed without any security check.

Memory handler

Procedure that can access (read/write) to Logical Block

4 SYMBOLS AND ABBREVIATIONS

ECU

Electronic Control Unit referred to as Server in ISO documents

N/A

Not Applicable

NRC

Negative Response Code. See document [1]

5 REQUIREMENTS TERMINOLOGY

The following terminology is used to define the applicability of each requirement in this document.

- The word **Must** in the text means legislative or regulatory requirements (e.g. Health and Safety).
- The word **Shall** in the text means a mandatory requirement.
- The word **Should** in the text means a recommendation or advice on implementing a requirement. Such recommendations or advice are expected to be followed; unless justified reasons are stated for not doing so.
- The word **Will** in the text means an intention or a provision, in connection with a requirement.
- The word **Can** in the text means a “user defined” requirement. The OEM designer shall choose how he wants to implement this requirement.

For our use, requirements are noted as follows:

Ref Version: Title	
Description	
<u>ATT Comments:</u>	Old reference:
	Genre:
	Validation way
<u>ATT Validation Method:</u>	
<u>ATT Flexibility:</u>	
<u>ATT Change Type:</u>	
<u>End Req</u>	

Ref: is a unique requirement identifier, which respects the following description:

[REQ_DIAG_Application_Number]

- **Domain:** Communication, Reprogramming, Application, Fault Manager, Archi
- **Number:** incremental number (for example 10 by 10 in case of add of new requirement in the chapter)

Version: The current version of the requirement

Title: is the descriptive of the requirement.

ATT_Flexibility:

F0 = no flexibility, requirement has to be met (must)

F1 = low flexibility, requirement barely negotiable (want, shall)

F2 = high flexibility, negotiable requirement (can, could, should, nice to have, will)

NA = Not applicable

ATT_Comment: note, remarks, example

- **Validation way:** dedicated to ECU designer, RTT ...
- **Old reference:** reference used before DOORS
- **Genre:** Generic for all ECU or specific for one ECU.

ATT_Change Type: The state of the requirement compared to the previous release (Unchanged, Modified, Created)

6 REQUIREMENTS

6.1 General

REQ_DIAG_Reprogramming_4010 a: Reset for bootloader

The boot-loader must support reset at any time.

ATT_Comments: **Validation way:** RTT
old reference : STD-PRG4-TS.0300
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4020 a: Procedures in bootloader

The boot-loader must contain no other procedures than described into this document to modify data contained in the Logical Blocks.

ATT_Comments: **Validation way:** dedicated to ECU designer
old reference : STD-PRG4-TS.0301
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4021 a: Forbidden in reprogramming

The area which is targeted for the digest calculation must not be modified after reprogramming

ATT_Comments: Do not modify the digest area to calculation by incidental work
Validation way: dedicated to ECU designer
old reference : N/A
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4030 a: SECURITY_STATE in bootloader

In boot-loader, if the value of SECURITY_STATE is not written specifically in the requirement (eg : write digest), all services work whatever the value of SECURITY_STATE.

ATT_Comments: *It's not necessary to unlock ECU to use some services (eg : read digest, read fingerprint...)*
Validation way: RTT
old reference : STD-PRG4-TS.0302
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4035 a: Protection of bootloader integrity

The boot-loader logical blocks shall not be modified by the applicative software.

ATT_Comments: *Example: a DID located in the boot-loader logical blocks shall not be modified by the applicative software.*

Validation way: dedicated to ECU designer

old reference : SN/A

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

Code that is able to modify memory must be protected against unexpected execution.

6.2 Boot-loader states

Boot-loader states are either DTC statuses or internal Boot-loader states.

6.2.1 Boot-loader DTCs

REQ_DIAG_Reprogramming_4040 a: DTC format in bootloader

Each DTC of the Boot-loader must be coded using 3-byte format defined into ISO 15031-6 norm.

ATT_Comments: *2 bytes for baseDTC and 1 byte for failure type. 0 for failure type should not be used.*

Validation way: RTT

Old reference : STD-PRG4-TS.0001

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

REQ_DIAG_Reprogramming_4050 a: DTC for validity of data associated with each Logical Block

Each Logical Block is associated to a DTC(Logical Block ID, Programming failure/Not programmed). This DTC provides information on the validity of the data contained into the logical block. Its ISO 15031-6 failure type is 0x51.

ATT_Comments: *This subtype is used by the control module to indicate that programming is required.*

Validation way: RTT

Old reference : STD-PRG4-TS.0002

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

REQ_DIAG_Reprogramming_4060 a: DTC for integrity of physical memory associated with each Logical Blocks

Each Logical Blocks are associated with DTC (Logical Block ID, system internal failures/General memory failure). This DTC provides information on the integrity of physical memory. Its ISO 15031-6 failure type is 0x42

ATT_Comments: *This subtype is used by the control module to indicate that the ECU is subject to a memory hardware failure.*

Validation way: RTT

Old reference : STD-PRG4-TS.0003

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

REQ_DIAG_Reprogramming_4070 a: Test status since last clear

Each DTC provides the information that it has not been tested since last clear.

ATT_Comments: Each DTC of the Boot-loader must support the status testNotCompletedSinceLastClear

Validation way: RTT

Old reference: STD-PRG4-TS.0004

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

REQ_DIAG_Reprogramming_4080 b: DTC 0x06 - algorithm based failures

The Boot-loader must implement at least one DTC of failure type "algorithm based failures" (0x06 code)

ATT_Comments: *In case of ECU which support plural Logical Blocks, and if compatibility verification is necessary between Logical Blocks, then ECU has to support Global DTC for this failure type.*

In case of ECU with support one logical block with one segment, the DTC shall be implemented but with no functional test associated. It will behave like a regular DTC, and when the routine for the DTC will be executed the DTC status will be set to 00.

Refer to annex A.2, A.3 and A.4

Validation way: RTT

Old reference: STD-PRG4-TS.0005

ATT_Flexibility:

F0

ATT_Change Type:

Modified

End Req

REQ_DIAG_Reprogramming_4100 a: Forbidden DTC

DTC different of those described above are forbidden in boot-loader

ATT_Comments: *To simplify the mechanism of reprogramming tool.*

Validation way: RTT

Old reference: STD-PRG4-TS.0007

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

6.2.2 Counters

Table

Counter name	Comment
LOGICAL_BLOK_REPROG_CNTR(Logical Block ID)	Each ECU can contain several Logical Blocks. The counter is the number of remaining possible update operations on the Logical Block

REQ_DIAG_Reprogramming_4110 a: DTC supported for a memory with programming limit

In case the Logical Block is contained into a memory with a programming limit, the DTC(Logical Block ID, system internal failures/General memory failure) must be supported.

The interface BOOT_GET_LOGICALBLOCK_INFO reports only the counters less than 126. See UDS norm.

ATT_Comments: *BOOT_GET_LOGICALBLOCK_INFO returns:126 - LOGICAL_BLOK_REPROG_CNTR(Logical Block ID).
e.g. "In case of the limit counter of the re-programming times is defined as 10, the initial value of "BOOT_GET_LOGICALBLOCK_INFO" will be returned as 117."*

Validation way: RTT

Old reference: STD-PRG4-TS.0010

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

Note: A Logical Block can be based on several memory types (several segments) with several programming limit. LOGICAL_BLOK_REPROG_CNTR(Logical Block ID) must reflect the lowest limit.

Into the following example, the Logical Block is composed of 3 segments of memory.

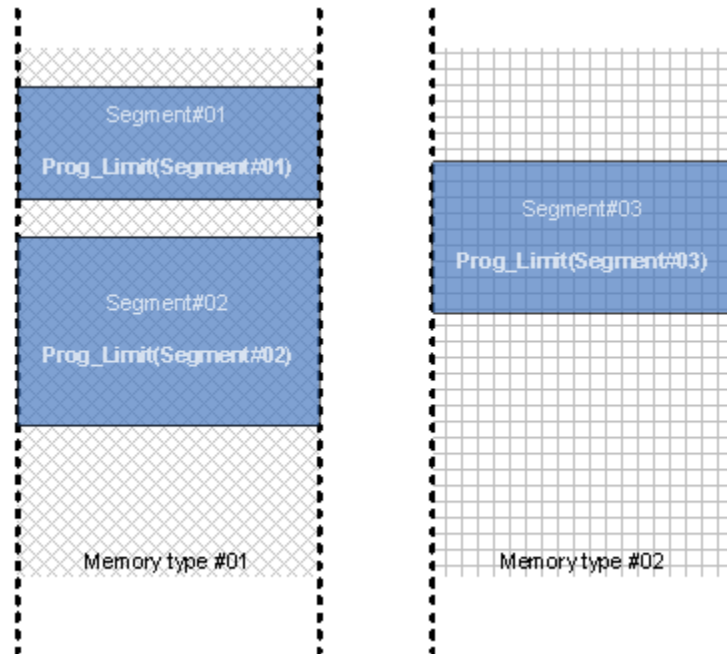


Figure 1 - LOGICAL_BLOK_REPROG_CNTR(Logical Block ID)

In this case, LOGICAL_BLOK_REPROG_CNTR(Logical Block ID) should be the minimum number between Prog_Limit(Segment#01), Prog_Limit(Segment#02) and Prog_Limit(Segment#03)

REQ_DIAG_Reprogramming_4120 a: BOOT_REQ_DOWNLOAD refused

In case the Logical Block is contained into a memory with a programming limit, for each Logical Block, when the LOGICAL_BLOK_REPROG_CNTR(Logical Block ID) reach the value 0, BOOT_REQ_DOWNLOAD is refused. In this case, NRC 0x70 should be used.

ATT_Comments:

Validation way: RTT

Old reference: STD-PRG4-TS.0011

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

REQ_DIAG_Reprogramming_4130 a: LOGICAL_BLOK_REPROG_CNTR not implemented

In case the Logical Block is contained into a memory without a programming limit, LOGICAL_BLOK_REPROG_CNTR(Logical Block ID) is not implemented.

ATT_Comments:

Case for E²PROM, Hard drives or case where limit is judged sufficient for the lifetime of the system.

Validation way: RTT

Old reference: STD-PRG4-TS.0012

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

REQ_DIAG_Reprogramming_4140 a: Decrementing of Prog_Limit for segment update

Prog_Limit(Segment#x) is decremented before each segment update.

ATT_Comments: *If Logical Block is contained into memory that needs erasing, Prog_Limit(Segment#x) is decremented before erasing. It is decremented after reprogramming in other cases.*

Validation way: RTT

Old reference: STD-PRG4-TS.0013

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End_Req

REQ_DIAG_Reprogramming_4150 a: Decrementing of Prog_Limit for segment erasing

If segment erasing is needed for reprogramming, Prog_Limit(Segment#01) (the first segment of the request download area) is decremented as part of executing BOOT_REQ_DOWNLOAD.

ATT_Comments: **Validation way:** RTT

Old reference: STD-PRG4-TS.0014

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End_Req

6.2.3 Internal states

Table

State name	Available states	Comment
SECURITY_STATE	LOCK	In this state, it's not possible to modify the digest of a Logical Block or to change data contained into a Logical Block that matches with its digest.
	UNLOCK	In this state, there are no restrictions regarding the services available.
MEMORY_HDLR_STATE	OPEN	Boot-loader is ready to receive downloaded data and to store it into its memory.
	CLOSED	Boot-loader is not ready to receive downloaded data and to store it into its memory.

REQ_DIAG_Reprogramming_4160 a: SECURITY_STATE for exit of programming session

SECURITY_STATE is set to LOCK when exiting the Programming session.

ATT_Comments: **Validation way:** RTT
Old reference: STD-PRG4-TS.0020
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4170 a: SECURITY_STATE for initialization step on boot

SECURITY_STATE is set to LOCK when on the Boot Initialization step.session

ATT_Comments: *Default value for SECURITY_STATE state is LOCK.*
Validation way: RTT
Old reference: STD-PRG4-TS.0021
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4180 a: MEMORY_HDLR_STATE for exit of programming session

MEMORY_HDLR_STATE is set to CLOSED when exiting the Programming session.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference: STD-PRG4-TS.0022
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4190 a: MEMORY_HDLR_STATE for initialization step on boot

MEMORY_HDLR_STATE is set to CLOSED on the Boot Initialization step

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference: STD-PRG4-TS.0023
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4200 a: MEMORY_HDLR_STATE for initialization of Programming Session

In case memory handler is contained into the Boot-loader, MEMORY_HDLR_STATE is set to OPEN on the Programming Session initialization.

ATT_Comments:

Validation way: dedicated to ECU designer

ATT_Flexibility:

Old reference: STD-PRG4-TS.0024

ATT_Change Type:

F0

Unchanged

End Req

6.3 Boot-loader activity diagram

Boot-loader activity diagram is represented into Figure 2 and Figure 3

6.3.1 Entering Programming Session

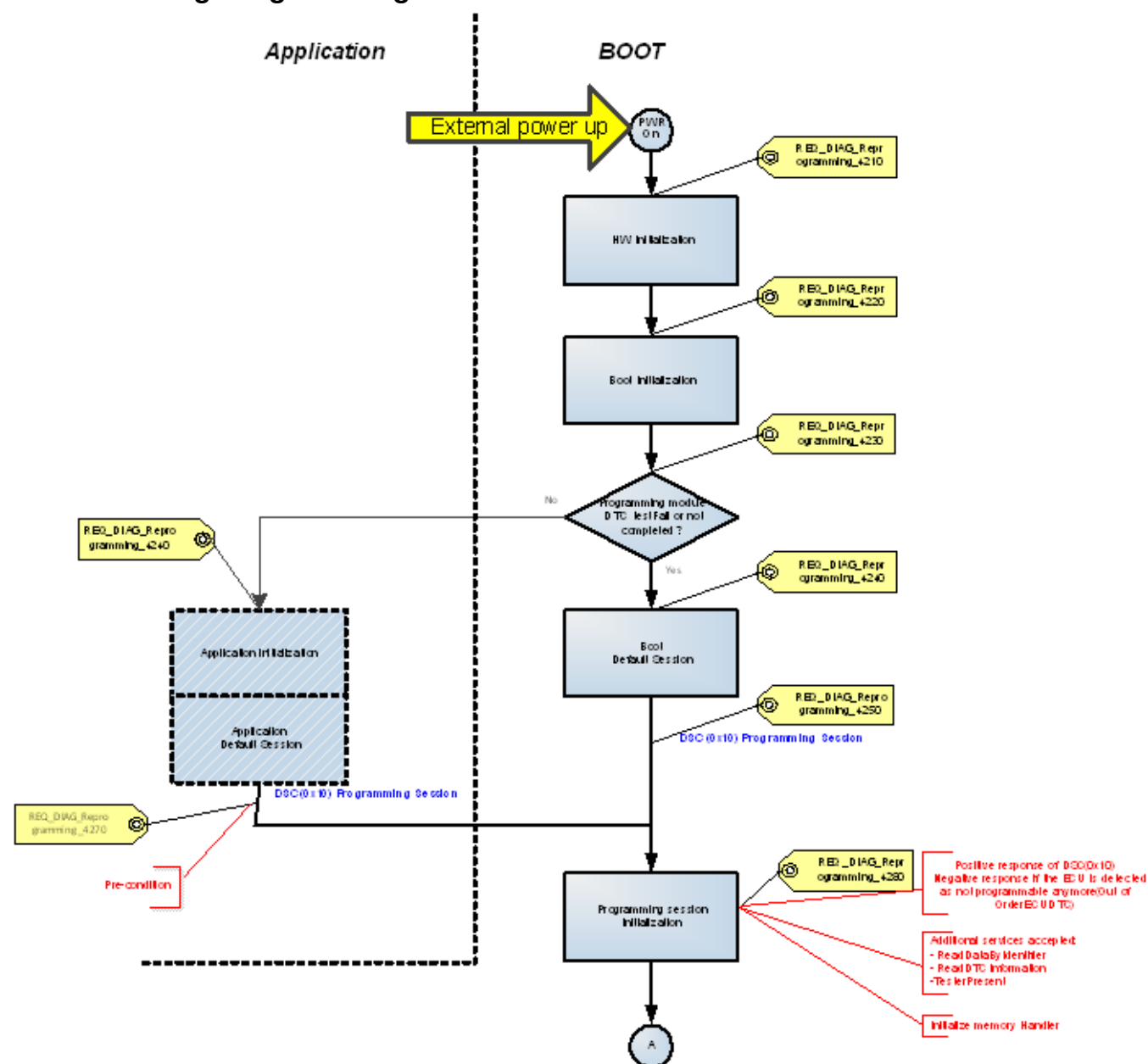


Figure 2 - Boot-loader activity diagram part #1 (1)

REQ_DIAG_Reprogramming_4210 a: Hardware initialization of bootloader

After external ECU power up, Boot-loader starts its hardware initialization.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference: STD-PRG4-TS.0101
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4220 b: Software initialization of bootloader

After ECU's hardware initialization, the Boot-loader starts its software initialization. The Boot-loader initialization contains only the steps necessary to handle the subsequent decision step of starting application. No DTC checks are done during this phase.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference: STD-PRG4-TS.0102
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4230 b: Verification of DTC status

After ECU's software Boot-loader initialization, the Boot-loader verifies the status of its DTCs

ATT_Comments: **Validation way:** RTT
Old reference: STD-PRG4-TS.0103
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4240 a: Conditions for application running

After Boot-loader DTC status check, if every DTC are tested and every tests returns no error, Boot-loader enable application running.

ATT_Comments: **Validation way:** RTT
Old reference: STD-PRG4-TS.0104
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4250 a: Default session of bootloader

After Boot-loader DTC status check, if at least a DTC is not tested or indicate an error, the Boot-loader goes into its Default Session. Any remaining boot-loader initialization must be performed at this step.

ATT_Comments:	Validation way: RTT
	Old reference: STD-PRG4-TS.0105
ATT_Flexibility:	F0
ATT_Change Type:	Unchanged

End Req

REQ_DIAG_Reprogramming_4260 b: Initialization of Programming Session in Boot-loader from Boot-loader default session

From Boot-loader default session, if Programming session is allowed, the Boot-loader Programming Session initialization is entered.

ATT_Comments:	Validation way: RTT
	Old reference: STD-PRG4-TS.0106
ATT_Flexibility:	F0
ATT_Change Type:	Unchanged

End Req

REQ_DIAG_Reprogramming_4270 b: Initialization of Programming Session in Boot-loader from Application ATT_Validation_Method:Test

From the application, ECU checks that all pre-conditions are satisfied and enter the Boot-loader Programming Session initialization.

These pre-conditions are to be specified, system by system, by the manufacturer ECU designers.

A negative response is sent to tool if a pre-condition is detected as not specified anymore.

ATT_Comments:	Validation way: RTT
	Old reference: STD-PRG4-TS.0107
ATT_Flexibility:	F0
ATT_Change Type:	Unchanged

End Req

REQ DIAG Reprogramming 4280 b: Response of DSC (0x10) to BOOT_PROG_SESSION

The positive response of DSC (0x10) to BOOT_PROG_SESSION is sent just after it been entered into. A negative response is sent if the ECU is detected as not programmable anymore (Out of OrderECUDTC).

Additional services are accepted :

- ReadDataByIdentifier
- ReadDTCInformation
- TesterPresent

ATT Comments:

Validation way: dedicated to ECU designer

Old reference: STD-PRG4-TS.0108

ATT Flexibility:

F0

ATT_Change Type:

Unchanged

End Req

6.3.2 Handling Programming procedures

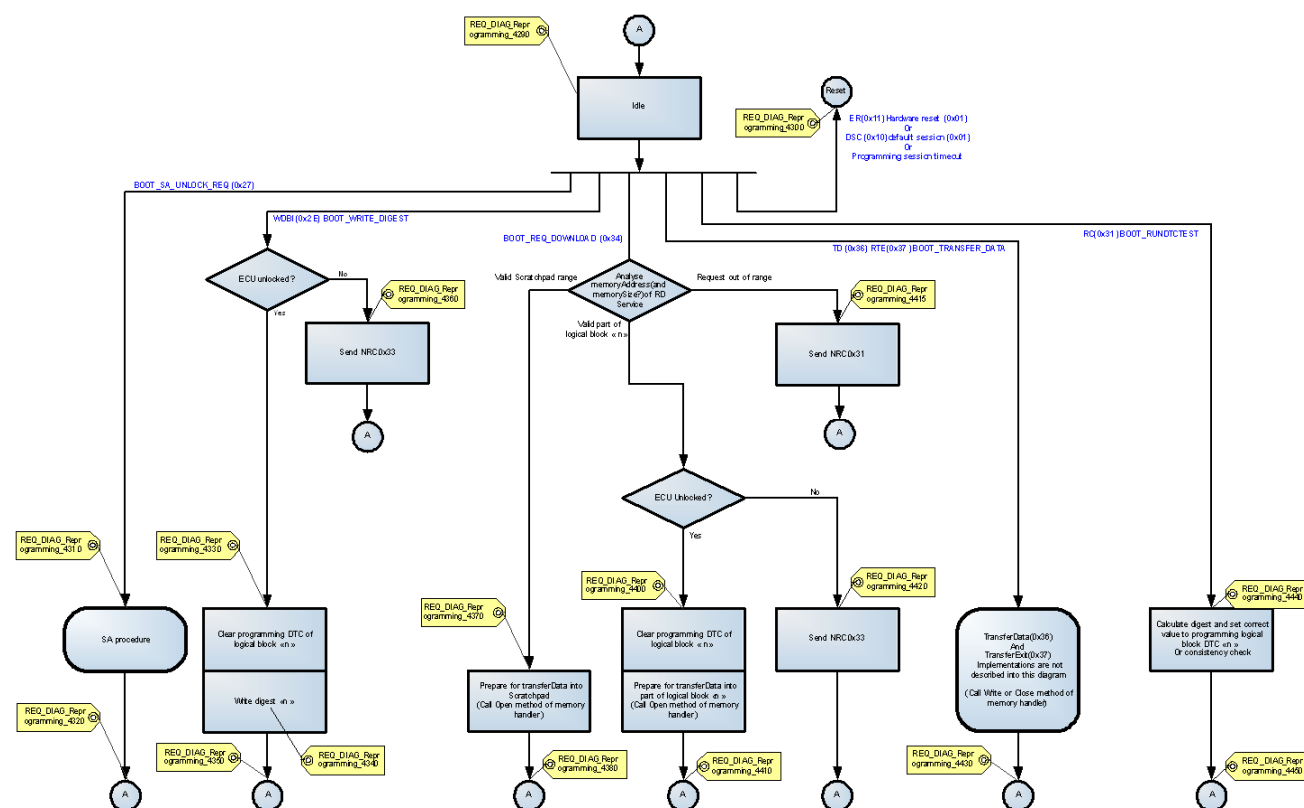


Figure 3 - Boot-loader activity diagram part #2

REQ_DIAG_Reprogramming_4290 b: Boot-loader behavior on idle state

During idle state, Boot-loader is just waiting next request.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference: STD-PRG4-TS.0220
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End_Reg

REQ_DIAG_Reprogramming_4300 a: Hardware reset of ECU

Exit of programming session starts a hardware reset of ECU. Exit of programming session is be done by SDS (0x10) default session, ECUReset (0x11) HW reset or reprogramming session time-out.

ECU must support SDS (0x10) and reprogrammation session time-out. ECUReset (0x11) HW is optional.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference: STD-PRG4-TS.0221
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End_Reg

REQ_DIAG_Reprogramming_4310 a: Start of unlocking security access procedure

In idle state, calling the BOOT_SA_UNLOCK_REQ interface starts the unlocking security access procedure.

ATT_Comments: **Validation way:** RTT
Old reference: STD-PRG4-TS.0201
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End_Reg

REQ_DIAG_Reprogramming_4320 a: Idle state of bootloader after security access

When security access procedure is over, whatever is the SECURITY_STATE, the Boot-loader goes back to its Idle state.

ATT_Comments: **Validation way:** RTT
Old reference: STD-PRG4-TS.0212
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End_Reg

REQ_DIAG_Reprogramming_4330 a: BOOT_WRITE_DIGEST with SECURITY_STATE unlocked

In idle state, calling the BOOT_WRITE_DIGEST forces the Boot-loader to check its SECURITY_STATE. In case it is into the state UNLOCK:

- The associated DTC state is cleared, this mean that its states indicates that the test has not been passed.
- The digest has to be written into ECU nonvolatile memory, outside of logical blocks area. Refer to "Figure 4 - Logical Block Digest"

When all the above steps are accomplished, the positive response to BOOT_WRITE_DIGEST is returned.

ATT_Comments: Validation way: RTT
Old reference: STD-PRG4-TS.0202
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4340 a: Digest

Each Logical Block is associated to a 16 bits DataIdentifier. Its dataRecord contains the Digest expected value. The value of the DataIdentifier is the 16 highest bits of the associated programmed/not programmed DTC. When all the above steps are accomplished, the positive response to BOOT_WRITE_DIGEST is returned.

ATT_Comments: Validation way: RTT
Old reference: STD-PRG4-TS.0203
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4350 a: Idle state of bootloader after writing digest

When writing digest procedure is over, the Boot-loader goes back to its Idle state.

ATT_Comments: Validation way: RTT
Old reference: STD-PRG4-TS.0213
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4360 a: BOOT_WRITE_DIGEST with SECURITY_STATE locked

In idle state, calling the BOOT_WRITE_DIGEST forces the Boot-loader to check its SECURITY_STATE. In case it is into the state LOCK, a negative response is sent as answer of BOOT_WRITE_DIGEST. Boot-loader turns then back to its Idle state.

ATT_Comments: Validation way: RTT
Old reference: STD-PRG4-TS.0219
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4370 a: BOOT_REQ_DOWNLOAD open MEMORY_HDLR_STATE

This requirement addresses only Boot-loaders that support scratchpad downloading.

In Idle state, calling BOOT_REQ_DOWNLOAD for a volatile memory download initialize memory handler. MEMORY_HDLR_STATE goes into OPEN state.

ATT_Comments: *No security check is necessary for this step.*

Validation way: RTT

Old reference: STD-PRG4-TS.0204

ATT_Flexibility: F0

ATT_Change Type: Unchanged

End_Reg

REQ_DIAG_Reprogramming_4380 a: Idle state of bootloader after scratchpad download

When preparing scratchpad download procedure is over, the Boot-loader goes back to its Idle state.

ATT_Comments: **Validation way:** RTT

Old reference: STD-PRG4-TS.0214

ATT_Flexibility: F0

ATT_Change Type: Unchanged

End_Reg

REQ_DIAG_Reprogramming_4400 c: Positive response to BOOT_REQ_DOWNLOAD when SECURITY_STATE is unlocked

In Idle state, if BOOT_REQ_DOWNLOAD is called for a non volatile memory download and if the SECURITY_STATE is into UNLOCK:

- The logical block associated DTC ("Programming failure/Not programmed" – FTB 0x51 and failure type "algorithm based failures" – FTB 0x06 if this one exists) state is cleared, this mean that its states indicates that the test has not been passed.
- Global DTC with failure type "algorithm based failures" if it exist, is cleared, this mean that its states indicate that the test has not been passed
- The memory handler is initialized accordingly. MEMORY_HDLR_STATE goes into OPEN state.

When all the above steps are accomplished, the positive response to BOOT_REQ_DOWNLOAD is returned.

ATT_Comments: **Validation way:** RTT

Old reference: STD-PRG4-TS.0208

ATT_Flexibility: F0

ATT_Change Type: Unchanged

End_Reg

REQ_DIAG_Reprogramming_4410 a: Idle state of bootloader after preparing download

When preparing download procedure is over, the Boot-loader goes back to its Idle state.

ATT_Comments: Validation way: RTT
Old reference: STD-PRG4-TS.0215
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4420 b: Negative response to BOOT_REQ_DOWNLOAD when SECURITY_STATE is locked

In Idle state, if BOOT_REQ_DOWNLOAD is called for a non volatile memory download AND IF the SECURITY_STATE is into LOCK, a negative response is sent as answer of BOOT_REQ_DOWNLOAD. Boot-loader turns then back to its Idle state.

ATT_Comments: Validation way: RTT
Old reference: STD-PRG4-TS.0209
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4430 a: Idle state of bootloader after transfer procedure

When transfer data procedure is over, the Boot-loader goes back to its Idle state.

ATT_Comments: *If transfer data was related to scratchpad, authenticity check can be performed. If it's not ok, transferred are erased.*
Validation way: RTT
Old reference: STD-PRG4-TS.0217
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4440 b: Idle state of bootloader after BOOT_RUNDTCTEST

In idle state, calling the BOOT_RUNDTCTEST forces Boot-loader to check if the associated Logical Block data matches with the corresponding digest or to check consistency (associated to logical block or global).

The corresponding DTC state is set according to the check result.

ATT_Comments: Validation way: RTT
Old reference: STD-PRG4-TS.0211
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4455 a: Idle state of bootloader after autocontrol procedure

When autocontrol procedure is over, the Boot-loader goes back to its Idle state

ATT_Comments:

Validation way: RTT

Old reference: STD-PRG4-TS.0218

ATT_Flexibility:

F0

ATT_Change Type:

Unchanged

End_Req

REQ_DIAG_Reprogramming_4455 a: Memory erasure

If reprogrammable memory needs erasing, this operation can be triggered on BOOT_REQ_DOWNLOAD or on BOOT_TRANSFER_DATA.

ATT_Comments:

Validation way: RTT

Old reference: STD-PRG4-TS.0218

ATT_Flexibility:

F0

ATT_Change Type:

Created

End_Req

6.4 Logical Block

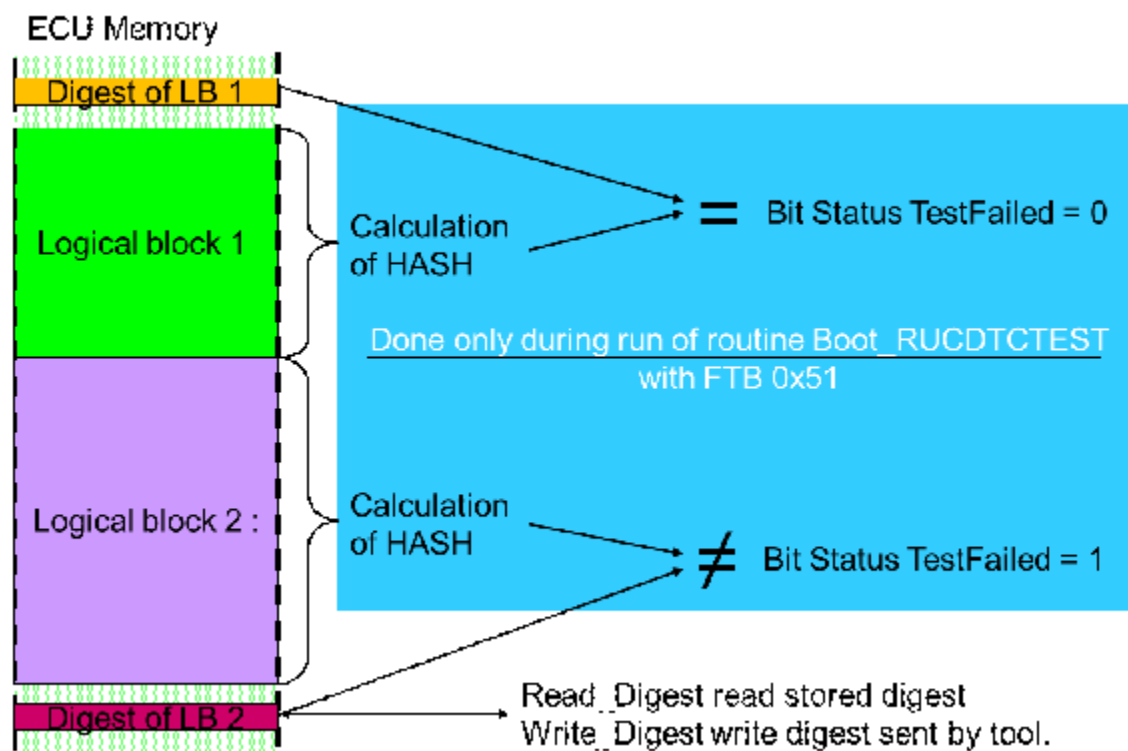


Figure 4 - Logical Block Digest

REQ_DIAG_Reprogramming_4460 a: Format of Logical Block's segments

Logical Block's segments address and size are statically defined.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference : STD-PRG4-TS.0400
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End_Req

REQ_DIAG_Reprogramming_4470 a: Digest characteristic

Digest is computed on the entire Logical Block's area.

ATT_Comments: *It's not mandatory that the programmed area covers the entire Logical Block.*
Validation way: dedicated to ECU designer
Old reference : STD-PRG4-TS.0401
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End_Req

REQ_DIAG_Reprogramming_4480 a: Cryptography for digest

Digest is computed using a standard cryptographic hash function.

ATT_Comments: **Validation way:** RTT
Old reference : STD-PRG4-TS.0402
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End_Req

6.5 Scratchpad and Memory Handler

REQ_DIAG_Reprogramming_4500 b: Default memory handler

IF ECU has scratchpad, default memory handler is loaded into scratchpad on Programming Session initialization.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference : STD-PRG4-TS.0500
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End_Req

REQ_DIAG_Reprogramming_4510 b: NRC 70

If BOOT_REQ_DOWNLOAD memory address is called for a non-volatile memory download and if the memory handler is not contained into scratchpad, NRC 70 is returned to the tool.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference : STD-PRG4-TS.0501
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4520 b: RAM Handler

ECU must check if signature of scratchpad match with authorized signature when TransfertExit (0x37) is called for a volatile memory.

ATT_Comments: *System supplier is responsible for providing signed memory handling routines.*
Validation way: dedicated to ECU designer
Old reference : STD-PRG4-TS.0502
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4530 b: Reload of default memory handler into scratchpad

If scratchpad signature does not match with authorized signature, default memory handler is reload into scratchpad and MEMORY_HDLR_STATE goes into CLOSE state.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference : STD-PRG4-TS.0503
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4531 a: Load of memory handler into scratchpad

If scratchpad signature match with authorized signature, memory handler shall be loaded into scratchpad AND MEMORY_HDLR_STATE stays into OPEN state.

ATT_Comments: **Validation way:** dedicated to ECU designer
Old reference : N/A
ATT_Flexibility: F0
ATT_Change Type: Unchanged

End Req

REQ_DIAG_Reprogramming_4532 a: Positive answer for download

For an ECU with scratchpad, If BOOT_REQ_DOWNLOAD is called for a non-volatile memory download and if the SECURITY_STATE is into UNLOCK and if the memory handler is contained into scratchpad, a positive response to BOOT_REQ_DOWNLOAD shall be returned.

ATT_Comments: **Validation way:** dedicated to ECU designer

Old reference : N/A

ATT_Flexibility: F0

ATT_Change Type: Unchanged

End_Req

Physical Memory Handler interface:

- Open (param: memoryAddress and memorySize, return: ok or NRC)
- Write (transferRequestParameterRecord)
- Close.

Switch between implementations is performed on RD service, depending on memory address.

Interface implementations:

- Default Handler: resident in boot. Stub (returns NRC 22 on Open)
- Flash write capable routines (resident or contained in Scratchpad).
- RAM Handler: resident in boot, write into Scratchpad.

In order to not make too complex programming data description (eg. ODX FLASH files), it is recommended to keep a logical block in a physical memory that is handled by a single memory handler.

6.6 Delivery State of ECU

2 Cases :

Case 1 : ECU is operational. Application is running

Case 2 : ECU is not operational. Application is not running and Bootloader is running because ECU is partly flashed by supplier (some logical blocks are intentionally left empty; ex: logical blocks for Calibration).

REQ_DIAG_Reprogramming_4540 a: Characteristic of an operational ECU when delivered in OEM plant/after-sales

An operational ECU must be delivered in OEM plant/after-sales with all Logical block **flashed by supplier**. All corresponding Digest are written and all corresponding Bootloader DTC are tested and succeed (Status byte = 0x00).

ATT_Comments: **Validation way:** dedicated to ECU designer

Old reference: STD-PRG4-TS.0050(1)

ATT_Change Type: Unchanged

End_Req

Example: Operational ECU with 2 logical blocks (FD01 and FD02) and global DTC

ECU Support		
Base DTC	DTC FTB	DTC Status
FD01 *Flashed by supplier	0x51	0x00 (Succeed)
	0x42	0x00 (Succeed)
	0x06	0x00 (Succeed)
FD02 *Flashed by supplier	0x51	0x00 (Succeed)
	0x42	0x00 (Succeed)
	0x06	0x00 (Succeed)
Global	0x42	0x00 (Succeed)
	0x06	0x00 (Succeed)

REQ_DIAG_Reprogramming_4550 a: Characteristic of a not operational ECU when delivered in OEM plant/after-sales

A not operational ECU (partly flashed by supplier; ex : Calibration is absent from ECU and will be flashed in OEM plant/after-sale) must be delivered in OEM plant/after-sales with:

- For Logical block **flashed in supplier plant** (Not flashed in OEM plant).
All DTC are tested and succeed (Status byte = 0x00) and with all corresponding Digest are written.
Refer to "Example: FD01 (SoftWare)"
- For Logical block **flashed in OEM plant/** after-sales (Not flashed in supplier plant)
DTC with FTB 0x51 are tested and failed (Status byte = 0x01) and an empty digest for logical block.
DTC with FTB 0x06 are tested and failed (Status byte = 0x01) Digest value (value stored in memory; readable by service \$22) must be different of calculated Digest of logical block area.

Refer to "Example: FD02 (Calib)"

ATT_Comments:

Validation way: dedicated to ECU designerr

Old reference: STD-PRG4-TS.0051(1)

ATT_Change Type:

Unchanged

End Req

Example: Not operational ECU-with 2 logical blocks (FD01 for software and FD02 for Calibration) and global DTC

ECU Support		
Base DTC	DTC FTB	DTC Status
FD01 (SoftWare) *Flashed in supplier plant	0x51	0x00 (Succeed)
	0x42	0x00 (Succeed)
	0x06	0x00 (Succeed)
FD02 (Calib) * Not flashed in supplier	0x51	0x01 (failed)
	0x42	0x00 (Succeed)
	0x06	0x01 (failed)
Global	0x42	0x00 (Succeed)
	0x06	0x01 (failed)

7 DELETED REQUIREMENTS

REQ_DIAG_Reprogramming_4490 issue a

8 Annex A: Supported Failure type

Usage of FTB :

Case 1 : supplier has defined only one logical block → supplier refers to annex A.1 and A.4 and ignore A.2 and A.3

Case 2 : supplier has defined more than one logical block → supplier has to define compatibility

Case 2.1 : supplier has defined compatibility (because it's a vehicle or supplier requirement)

→ supplier refers to annex A.3 and A.4 and ignore A.1 and A.2

Case 2.2 : supplier has not defined compatibility (because it's not a vehicle or supplier requirement) → supplier refers to annex A.2 and A.4 and ignore A.1 and A.3.

8.1 A.1: Supported Failure type for ECU with ONLY one logical block

Table

Failure Type	DTC
	Logical Block
0x51 (Programming failure/Not programmed)	M
0x42 (system internal failures/General memory failure)	M
0x06 (algorithm based failures)	M

8.2 A.2: Supported Failure type for ECU with multiple logical block and no global DTC

Table

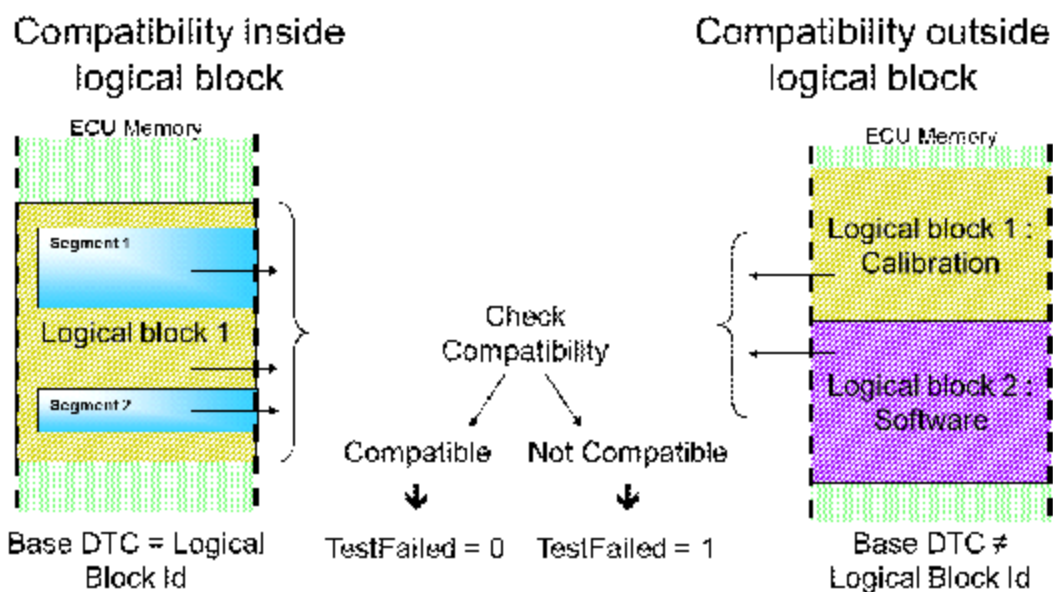
Failure Type	DTC
	Logical Block
0x51 (Programming failure/Not programmed)	M
0x42 (system internal failures/General memory failure)	M
0x06 (algorithm based failures)	M

8.3 A.3: Supported Failure type for ECU with multiple logical block and global DTC

Table

Failure Type	DTC	
	CVT. for Logical Block	CVT. For Global DTC
0x51 (Programming failure/Not programmed)	M	F
0x42 (system internal failures/General memory failure)	M	P
0x06 (algorithm based failures)	P	M

8.4 A.4: Management of Boot Loader DTC type 0x06



Global DTC: DTC not attached to a logical block but to a memory space.
DTC 0x06 must not include result of matching test (0x51).

9 Annex B: conversion of the DTC status

9.1 B.1: Management of DTC with FTB 0x51 or 0x06

Table

FT	Before execution of REQ_DIAG_Reprogramming_4330		After execution of REQ_DIAG_Reprogramming_4330	
	Bit4	Bit0	Bit4	Bit0
0x51	0	0	1	0
	1	0	1	0
	0	1	1	0
0x06	0	0	0	0
	1	0	1	0
	0	1	0	1

Table

FT	Before execution of REQ_DIAG_Reprogramming_4400		After execution of REQ_DIAG_Reprogramming_4400	
	Bit4	Bit0	Bit4	Bit0
0x51	0	0	1	0
	1	0	1	0
	0	1	1	0
0x06	0	0	1	0
	1	0	1	0
	0	1	1	0

Table

FT	Before execution of REQ_DIAG_Reprogramming_4440		After execution of REQ_DIAG_Reprogramming_4440	
	Bit4	Bit0	Bit4	Bit0
0x51	0	0	0	0 or 1
	1	0	0	0 or 1
	0	1	0	0 or 1
0x06	0	0	0	0 or 1
	1	0	0	0 or 1
	0	1	0	0 or 1

9.2 B.2: Management of DTC with FTB 0x42

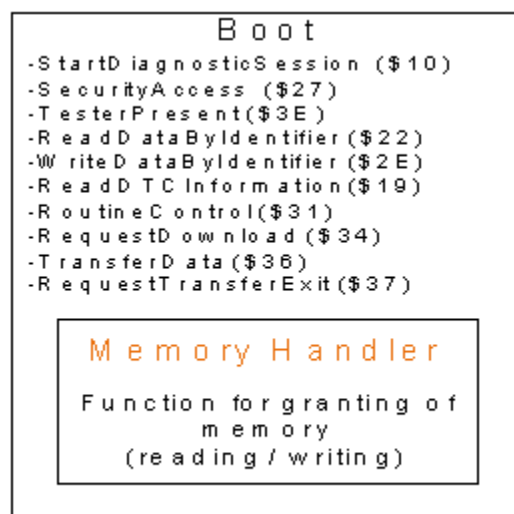
Requirement is REQ_DIAG_Reprogramming_4330, REQ_DIAG_Reprogramming_4400 or REQ_DIAG_Reprogramming_4440

FT	Before execution of requirement		After execution of requirement	
	Bit4	Bit0	Bit4	Bit0
0x42	0	0	0	0 or 1*
	1	0	0	0 or 1*
	0	1	0	1*

* 1 ==> BOOTLOADER detects an error during writing in non-volatile memory and error couldn't be corrected (need a replacement of ECU).

10 Annex C: Memory Handler management

10.1C1: 1st case ECU does not have scratchpad

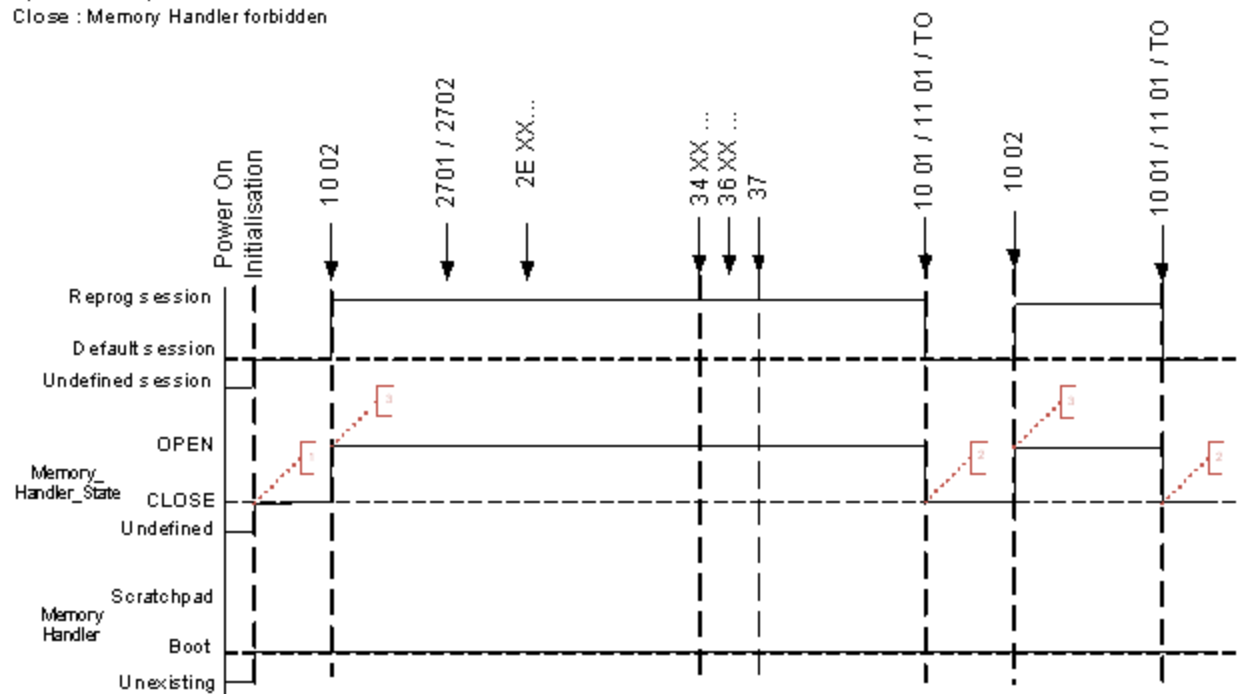


- In case of the ECU don't support Scratchpad, the memory Handler shall always be contained into the Bootloader.

Memory_Handler_State

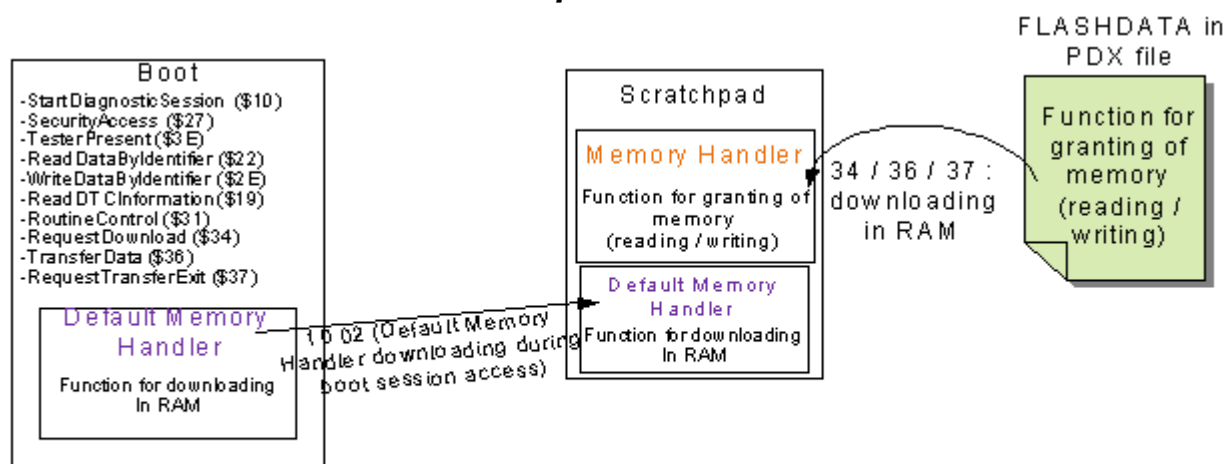
Open : Memory Handler authorized

Close : Memory Handler forbidden



- [1] REQ_DIAG_Reprogramming_4190
- [2] REQ_DIAG_Reprogramming_4180
- [3] REQ_DIAG_Reprogramming_4200

10.2C2: 2nd case ECU has scratchpad



OPEN Handler: This method is called WHEN the BOOT_REQ_DOWNLOAD (0x34) is started. It's authorize writing in memory (RAM/NVRAM)

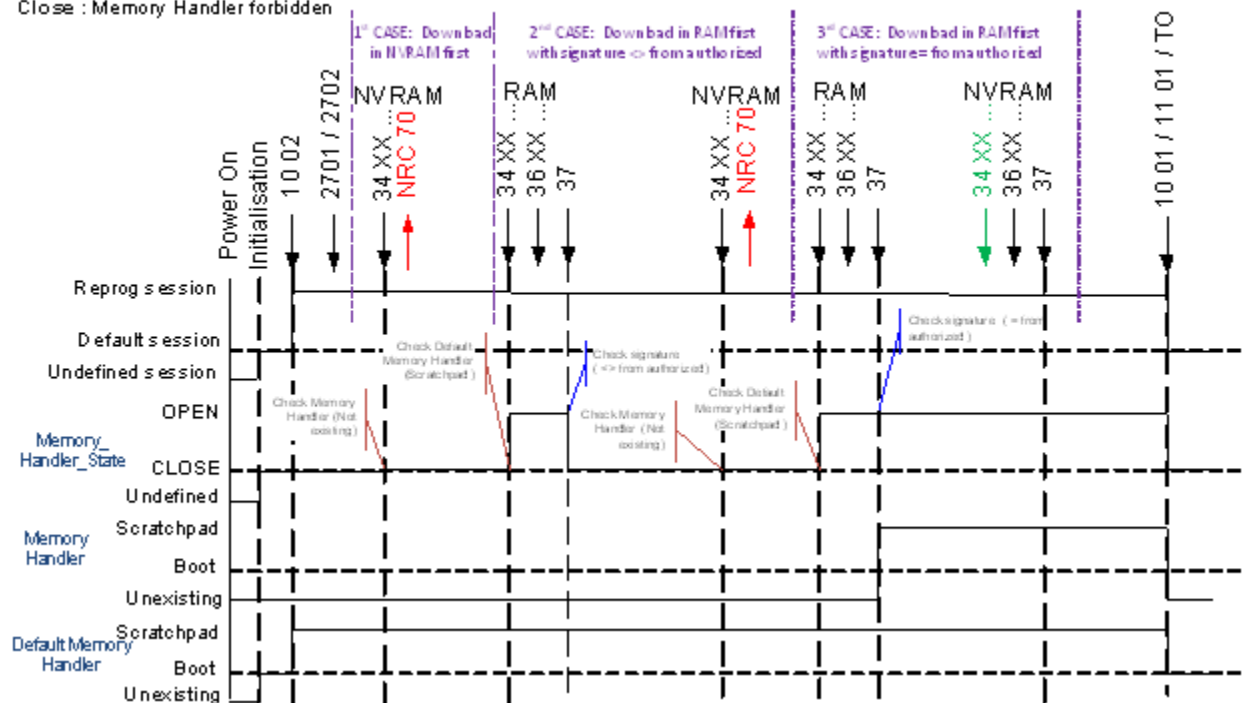
CLOSE Handler: This method is called WHEN the BOOT_TRANSFER_DATA (0x37) is started. It's forbid writing in memory (RAM/ NVRAM)

- **Overview**

Memory_Handler_State

Open : Memory Handler authorized

Close : Memory Handler forbidden

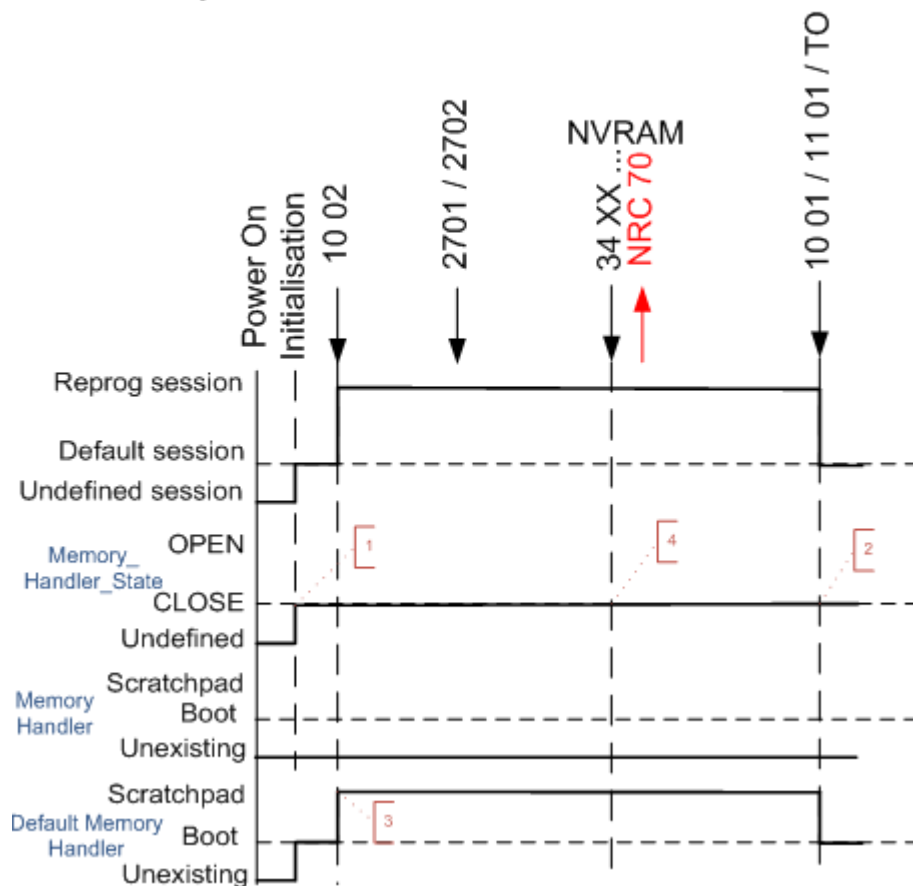


- 1st case: Download in NVRAM first

Memory_Handler_State

Open : Memory Handler authorized

Close : Memory Handler forbidden



[1] REQ_DIAG_Reprogramming_4190

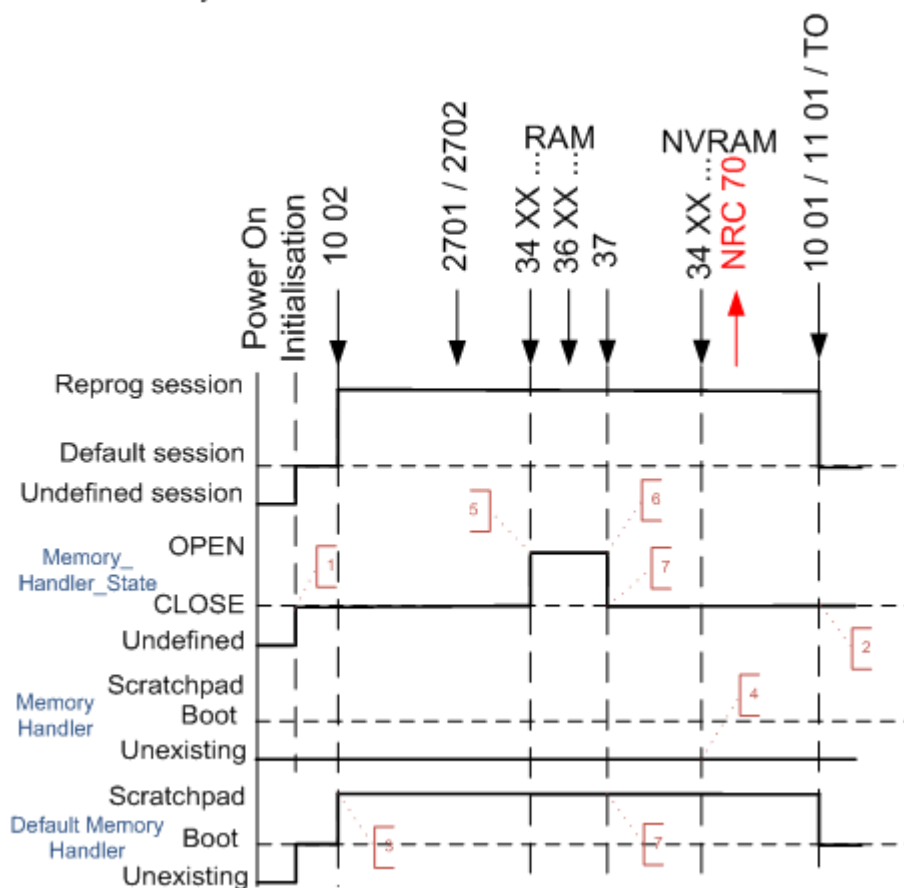
[2] REQ_DIAG_Reprogramming_4180

[3] REQ_DIAG_Reprogramming_4500

[4] REQ_DIAG_Reprogramming_4510

- 2nd case: Download in RAM first with signature <> from authorized

Close : Memory Handler forbidden



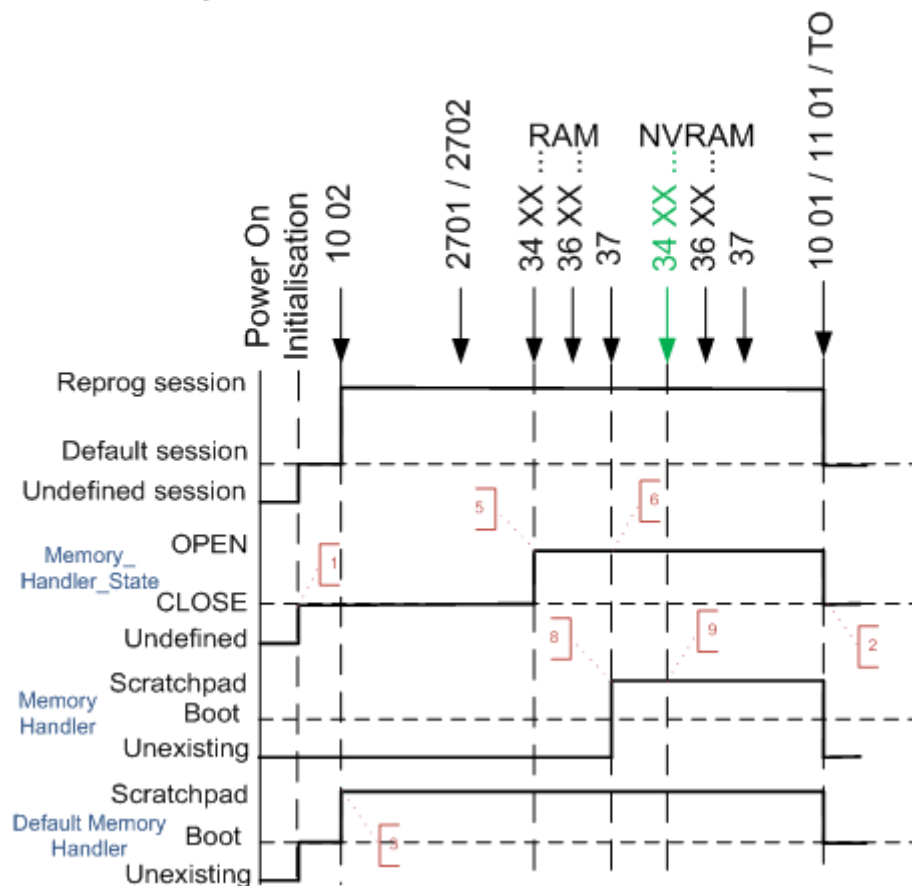
- [1] REQ_DIAG_Reprogramming_4190
- [2] REQ_DIAG_Reprogramming_4180
- [3] REQ_DIAG_Reprogramming_4500
- [4] REQ_DIAG_Reprogramming_4510
- [5] REQ_DIAG_Reprogramming_4370
- [6] REQ_DIAG_Reprogramming_4520
- [7] REQ_DIAG_Reprogramming_4530

- 3rd case: Download in RAM first with signature = from authorized

Memory_Handler_State

Open : Memory Handler authorized

Close : Memory Handler forbidden



[1] REQ_DIAG_Reprogramming_4190

[2] REQ_DIAG_Reprogramming_4180

[3] REQ_DIAG_Reprogramming_4500

[5] REQ_DIAG_Reprogramming_4370

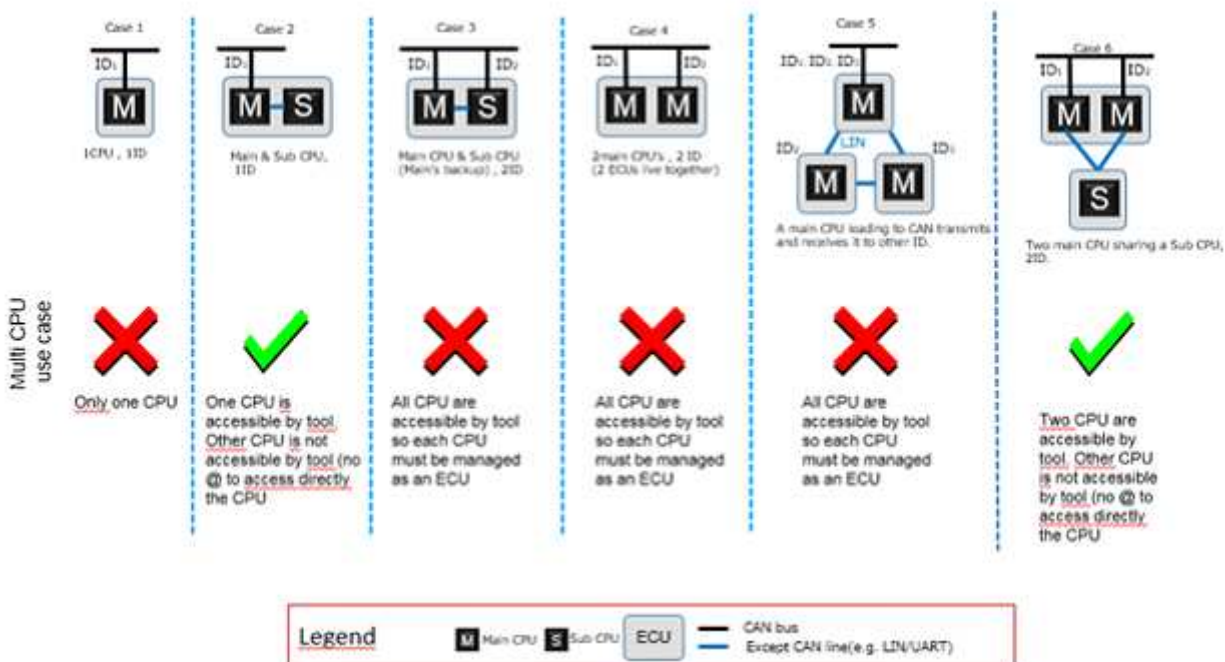
[6] REQ_DIAG_Reprogramming_4520

[8] REQ_DIAG_Reprogramming_4531

[9] REQ_DIAG_Reprogramming_4532

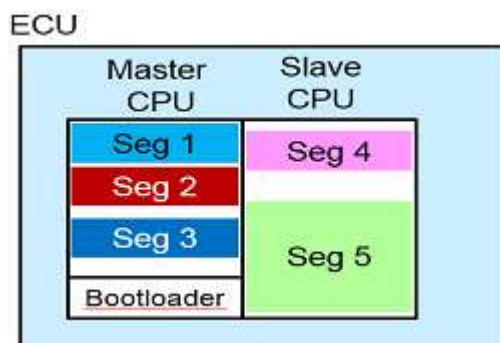
11 Annex D (Informative): Reprogrammation of Multi CPU

11.1D.1 Use cases to apply rules on multi CPU



Application of Multi CPU rules is applied only when CPUs are not accessible by Diag tool. A sub-part is the combination of a Master CPU accessible by tools and one (or several) sub CPU non accessible by tools like in case 2. In this case there is only one sub part in the ECU. When a sub CPU is shared between several masters CPU, it shall be associated with only one master CPU. Ex: in case 6, we shall only consider 2 sub parts for this ECU: M1/S1 and M2, or M1 and M2/S1. The cases multicore CPU and multi softwares on the same CPU are not in the scope of the modification.

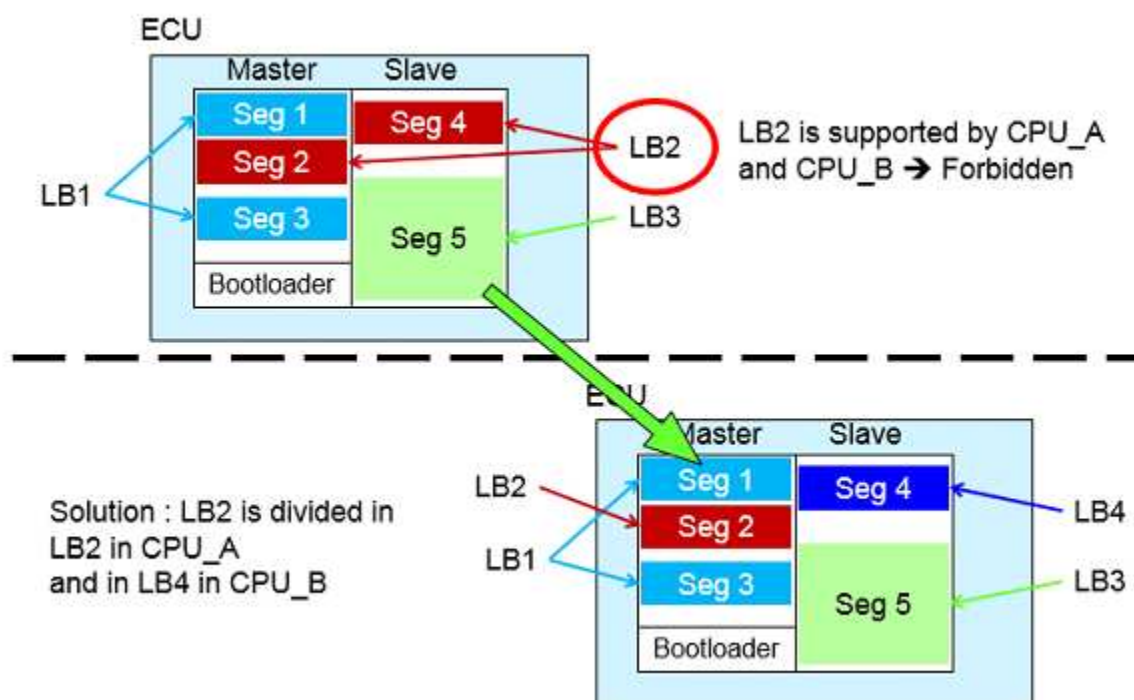
11.2D.2 Reprogrammation rules



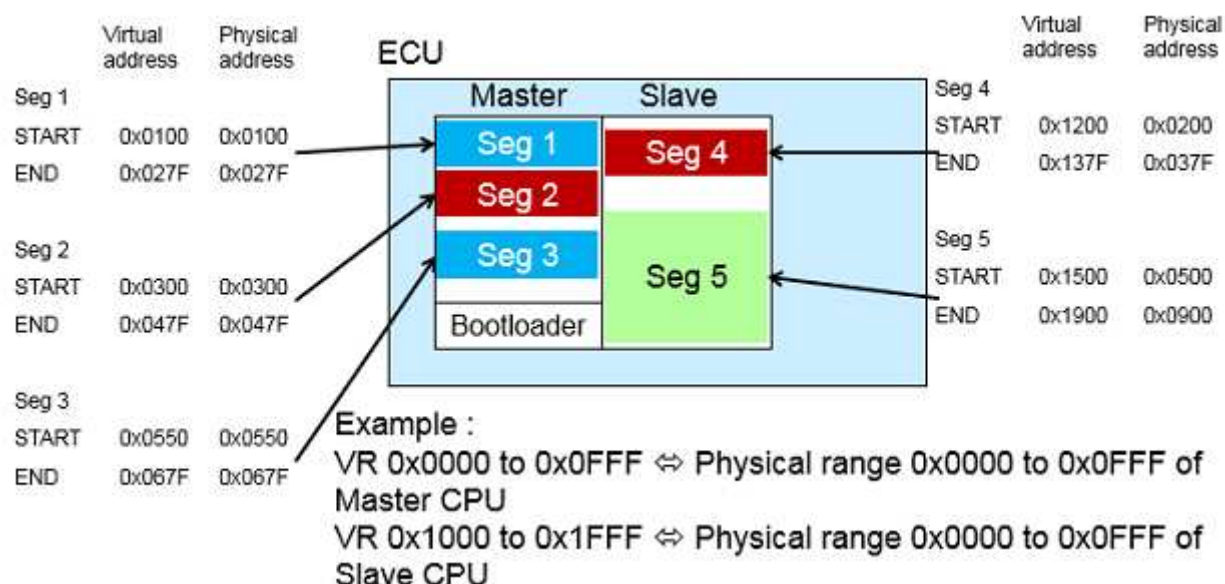
- One segment is located in only one CPU. No overlapping of 1 segment on 2 CPU.
- Each CPU contains one or several segments.
- In case of an ECU containing several CPU, the R/N bootloader is supported only by CPU accessible by diagnostic tools.
- Bootloader is able to access to all flashable segments whatever the localisation in CPUs.
- The master CPU of a sub part is responsible of the reprogramming, digest, DTC ... of its slave CPU.

Note:

- A segment is a part of memory with a Start Address and a size.
- A logical block has at least one segment.
- No overlapping of 1 logical block on different CPU. All segments of logical blocks (if more than one in logical block) must be on same CPU. Use of different CPU for the same logical block is forbidden.



- Digest and DTC have to be located on the CPU which supports Bootloader.
- Bootloader has to know which segment is addressed by using virtual address. Each CPU has a dedicated Virtual Range (VR) to allow switching between CPUs in bootloader.

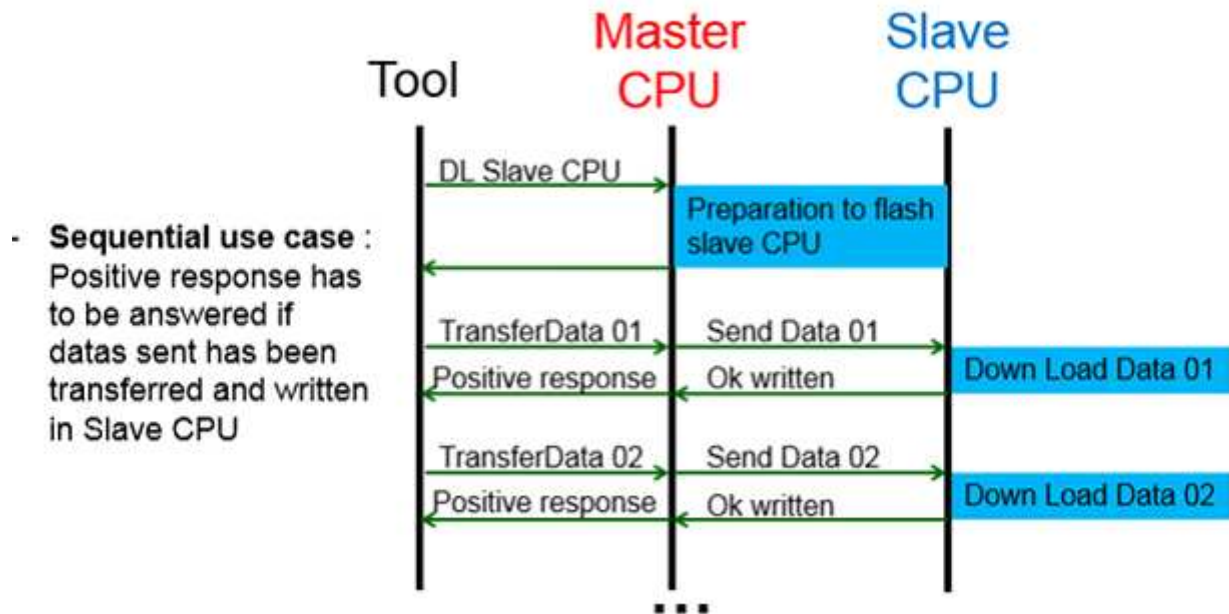


- If compatibility check is needed between CPUs, Global DTC FTB 0x06 is the result of compatibility check of all LB of all CPUs

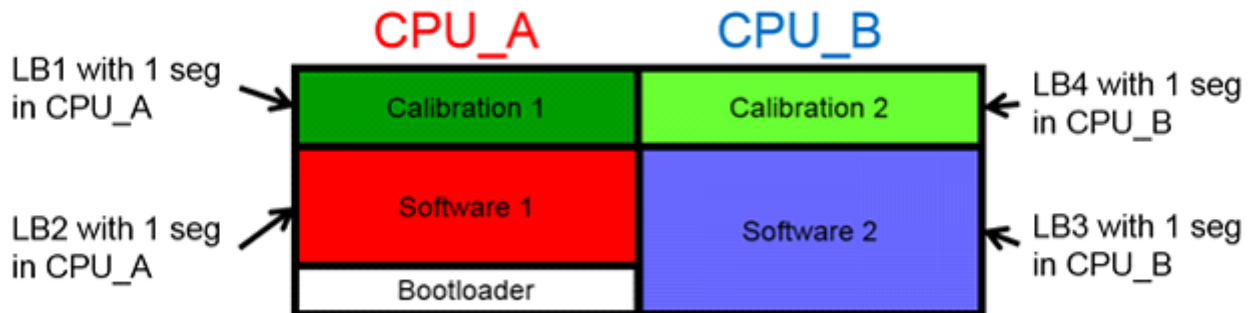
11.3D.3 Sequencing of transfer data

- Data to slave CPU are transferred from Master CPU during tool transferData processing.

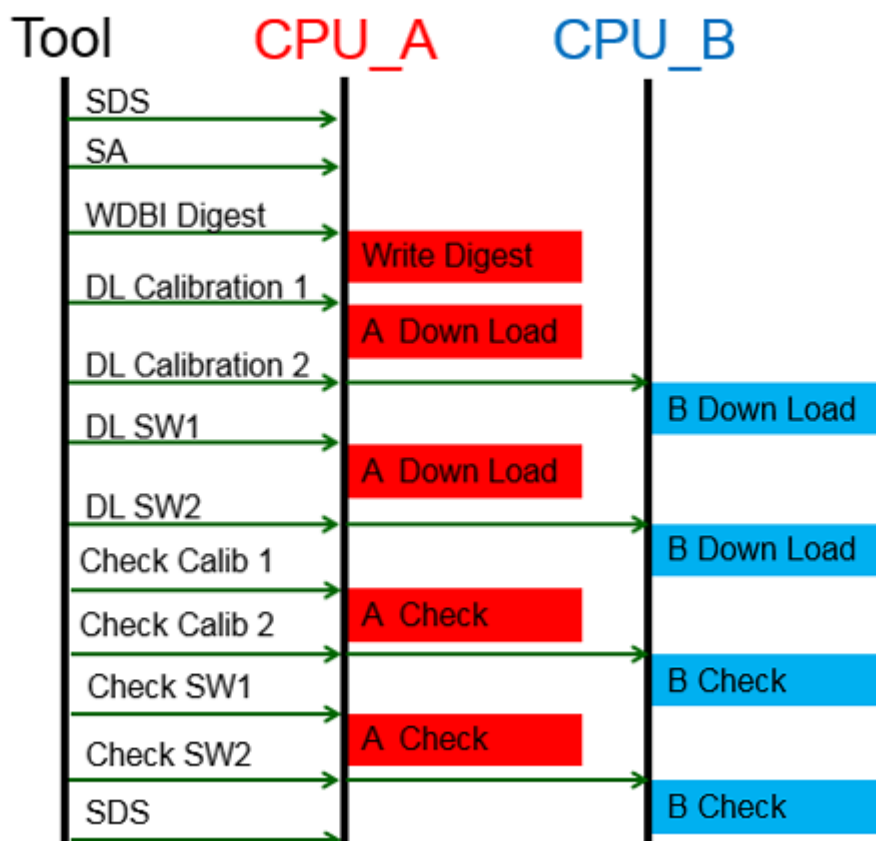
11.3.1 D.3.1 Sequential transfer



11.3.2 D.3.2 Example of the reprogramming of a Multi CPU ECU: 2 ECU/ 4 logical blocks



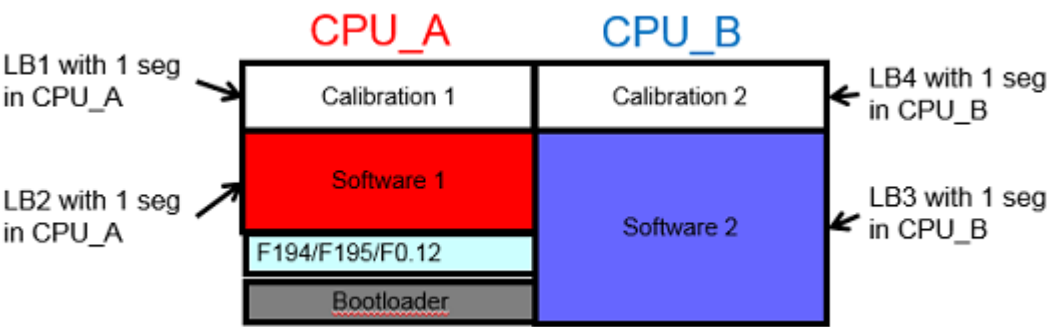
<Behavior of the Reprogram sequence (UDS)>

**11.4D.4 After reprogramming**

The software of the master CPU will be operational only if the software of the slave(s) CPU has/have the ability to become operational. As the master and the slave(s) CPU are considered as one part, this part should be operational only if all sub parts are.

11.5D.5 DID management

- The DIDs list in [REF 2] (RNDS-C-00073) shall be located on the CPU master of the sub part.
- The DIDs \$F1.88, \$F1.94, \$F1.95 and \$F1.A2 cover the whole sub part. Ex: \$F1.94 =Soft1 (Master) +Soft2 (slave).
- To avoid to have to download all the software (for Master and slave(s) CPU) at each modification, the DID \$F0.12, \$F1.94 and \$F1.95 will be located in an independent logical block in the Master CPU.



End of the document
